インジェクション脆弱性検知に必要な操作に関する実態調査

千田 忠賢[1,a)]　山口 大輔[2]　上川 先之[1,3]

概要：インジェクション脆弱性とは，悪意のある外部入力がプログラムの一部として解釈・実行される脆弱性であり，情報漏洩やデータの改ざんを引き起こし得る深刻な脅威である．その根絶に向けて，インジェクション脆弱性の検知に関する研究が広く行われている．これらの研究で広く用いられている手法では，外部入力を受け取る箇所（ソース）から，その入力がプログラムとして評価される箇所（シンク）に至るまでに適用される操作を取得し，それらを解析することで悪意のある外部入力をシンクまで到達させられるかを判定する．そのため，これらの手法による検知の精度は，ソースからシンクまでの過程で適用される操作をどれだけ漏れなく正確に捉えられるかに依存する．
本論文では，実世界で広く利用されている一方で，既存研究による対応が限定的な操作に着目し，それらがインジェクション脆弱性の検知精度に与える影響を明らかにするための調査を行う．具体的には，既存研究では文字列から文字列への変換を行う関数が広く解析対象とされている一方で，文字列から文字列リストへの変換を経由する操作への対応は限定的である．また，文字列操作には正規表現が広く用いられているが，既存の解析手法の多くは，先読み，後読み，後方参照などの拡張機能をサポートしていない．そこで本論文では，これらの操作や機能が実世界のプログラムにおいてどの程度利用されているかを調査する．さらに，その調査から得られた知見に基づき，今後のインジェクション脆弱性検知技術の発展に向けた技術的課題を整理する．

キーワード：インジェクション脆弱性，文字列制約解消

An Empirical Study of Real-World String Manipulations in Injection Vulnerability Detection

Nariyoshi Chida[1,a)]　Daisuke Yamaguchi[2]　Hiroyuki Uekawa[1,3]

Abstract: Injection vulnerabilities are serious threats in which malicious external input is interpreted and executed as part of a program, potentially leading to information leakage or data tampering. To overcome these threats, there are many existing works for detecting such vulnerabilities. A common approach used in these works is to collect the operations applied to external input along the path from a source, where the input enters the program, to a sink, where the input is interpreted as code, and analyze those operations to determine whether malicious input can reach the sink. Therefore, the accuracy of the approach depends on how completely and precisely they can model the operations applied between sources and sinks.
This paper investigates operations that are widely used in real-world programs but only partially supported by existing works, with the aim of clarifying their impact on the accuracy of injection vulnerability detection. In particular, while prior work has extensively analyzed functions that transform strings into strings, support remains limited for operations involving intermediate transformations from strings to lists of strings. Moreover, although regular expressions are widely used in string manipulations, many existing works do not support advanced features such as lookahead, lookbehind, and backreferences. We therefore examine how frequently these operations and features are used in real-world programs. Based on the findings, we further identify the technical challenges that must be addressed to advance future techniques for injection vulnerability detection.

Keywords: Injection vulnerability, string constraint solving

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

インジェクション脆弱性とは，悪意のある外部入力がプログラムの一部として解釈・実行される脆弱性である．ソフトウェアにインジェクション脆弱性が悪用できる形で含まれた場合，それを悪用されることでソフトウェア内の機密情報が外部に漏洩したりデータを改ざんされるなどの深刻な被害を引き起こし得る．この脅威を克服するため，ソフトウェア内に潜むインジェクション脆弱性を検知する技術に関する研究が広く行われているが，未だにインジェクション脆弱性の報告は絶えない．現に，National Vulnerability Database (NVD) [24] によると，2026 年に報告された全ての脆弱性のうち 12.69%がインジェクション脆弱[*1]に関するものであった．

本研究では，インジェクション脆弱性検知に関する多くの既存研究による進展があってもなお実世界においてインジェクション脆弱性の混入が絶えない原因解明を進める．特に，本研究ではインジェクション脆弱性検知に用いられる既存技術に焦点を当て，それらの技術が実世界のインジェクション脆弱性を精度良く検知する上で技術的に不足している点がないかという点で調査を行う．様々なインジェクション脆弱性検知技術が考案されているが，代表的な方法では大雑把には次の方法でインジェクション脆弱性の有無を検査する。まず，ソフトウェアにおける外部入力を受け取る箇所（ソース）と値をプログラムとして評価する箇所（シンク）に着目し，ソースから入力した値がシンクに到達するまでの過程で適用される操作を取得する．次に，取得した操作の解析し，入力する値を工夫することでシンクに脆弱性を悪用する値を到達させることが可能か判定する．そのため，この手法の精度は ① 値に適用された操作をいかに漏れなく取得できるか，そして ② 取得した操作をいかに正確に解析できるかという点に依存している．本研究では前者（すなわち，①）に着目する．すなわち，既存研究は外部から入力された値に適用された操作を漏れなく取得できているだろうか？

本問に対する答えを探るため，本研究では既存のインジェクション脆弱性検知技術が取得している操作と一般的な文字列操作で用いられる操作を比較し，既存技術では取得されていない操作について，それらが実世界でどれだけ外部入力に対して適用されているのか調査を行う．具体的には，(i) 多くの既存研究では主に文字列から文字列への関数を取得しているが，一般的な文字列操作においては，文字列から文字列への変換を行う際にはそれらのみではなく，文字列からリストへの変換やリストから文字列への変換など，豊富な種類の操作が用いられている．加えて，(ii) 文字列から文字列への操作に限定した場合であっても，そこで広く用いられている正規表現を伴う操作において，既存研究では先読み・後読み・後方参照などの拡張機能への対応は限定的である．これらの既存技術では対応していない操作が実世界でどれだけ利用されているのか確認し，インジェクション脆弱性を検知する上でどれだけ問題となっているのか考察する．

この調査を行うため，(i) に関して，我々は外部入力に対して適用される操作を広く取得するツールを実装した．そのツールを用いて実世界のウェブアプリケーションを探索し，外部入力に対して適用された操作をログとして取得し，取得したログを分析することで既存技術が対象としていると実世界で用いられる操作のギャップを探る．このようなログを取得するにはウェブアプリケーションに対して外部入力を与え実行する必要がある．本研究では，研究倫理の観点から，インジェクション脆弱性の中でもクライアント側で検知が完結する DOM 型 XSS を対象とする（cf. 第 6 章）．これは，XSS の中でも昨今 DOM 型 XSS に関する注目が高いことに加え，実世界のウェブアプリケーションに対して悪影響を与えることを避けつつ調査を実施することができると期待できるからである．次に，(ii) に関して，既存手法を用いてウェブアプリケーション内で利用されている文字列操作を取得し，取得した操作に用いられている正規表現を収集して分析を行った．これらの調査の結果として，既存技術では検査の対象に含めていない操作が実世界においては広く利用されていることを確認したため，この結果について報告し，インジェクション脆弱性検知技術の今後の発展の方向性について議論する．

2. 背景

本章では，DOM 型 XSS の検知技術を用いて既存のインジェクション脆弱性検知手法について概要を述べ，そこにおける課題を説明する．

2.1 DOM 型 XSS

DOM 型 XSS とはウェブアプリケーションの外部入力を受け取る箇所（以降ではソースと呼ぶ）から受け取った値が，文字列をプログラムとして評価する箇所（以降ではシンクと呼ぶ）に到達しプログラムとして評価される脆弱性である．ここで，ソースの例として `location.hash` や `document.referrer`，シンクの例として `element.innerHTML` や `eval` が挙げられる．ウェブアプリケーションに DOM 型 XSS が潜んでいる場合，攻

[1] NTT 社会情報研究所
NTT Social Informatics Laboratories
[2] NTT ソフトウェアイノベーションセンタ
NTT Software Innovation Center
[3] NTT セキュリティホールディングス
NTT Security Holdings
[a)] na.chida@ntt.com

[*1] インジェクション脆弱性の中でも特に本研究に関係する CWE-20 (Improper Input Validation) 及び CWE-79 (Cross-site Scripting) の報告件数であり，インジェクション脆弱性全体の報告件数はより多い．

© 2026 Information Processing Society of Japan

撃者はそれを悪用し，入力を工夫することでソースからシンクに脆弱性を悪用する文字列（例えば，<img src=1 onerror=alert('hack')> など）を到達させる．シンクはこれをプログラムとして評価してしまうため，これにより，例えば，攻撃者が入力した JavaScript のプログラムが実行され，情報漏洩やデータの改ざんが行われてしまう．

以下に DOM 型 XSS の脆弱性が潜む JavaScript のプログラムの断片を示す．このプログラムはウェブアプリケーションの中で利用されているものとする．

```
1 var hash = location.hash;
2 var parts = hash.split('/');
3 if (parts[1] === "compare") {
4   var reg = /^(?=.{1,100}$)[0-9a-z%-_ .]+$/i;
5   var items = parts.slice(2)
6           .filter((part) => reg.test(part));
7   titleElem.innerHTML = items.join(" vs ");
8 }
```

まず，1 行目では現在の URL の hash を取得し変数 hash に格納する．URL の hash とは，URL の末尾に出現する # とフラグメント識別子からなる文字列である．[*2] 例えば，URL が https://example.com/#/compare/iPhone17/Pixel11 なら，hash は #/compare/iPhone17/Pixel11 である．ここでは説明を簡略化するため，location.hash にはブラウザによる URL エンコーディングが適用されていないものとする．そのため，変数 hash には URL の hash がそのまま格納される．次に，2 行目では hash に格納された文字列を文字 / で分割して文字列のリストに変換し，変数 parts に格納する．例えば，hash に格納された値が #/compare/iPhone17/Pixel11 であるなら，parts に格納されるリストは [#, compare, iPhone17, Pixel11] である．

4 行目では parts の compare より後ろの各要素が意図した文字列のパターンに一致するかを確認するための正規表現 reg を定義している．具体的には，この正規表現における ^···$ は ··· の表現が文字列全体に一致するかを確認する．先読み (?=.{1,100}$) では文字列の長さが 1 以上 100 以下であることを確認し，範囲文字の繰り返し [0-9a-z%-␣.]+ は文字列が英数字 0，···，9，a，···，z（大小区別なし），および記号 %，-，_，␣（空白文字），. のいずれかの文字のみで構成されていることを確認することを意図している．ただし，後述の通り，この範囲文字の繰り返しにはバグがあり，この説明の通りになっていないことに注意されたい．

続く 5 行目では parts の compare より後ろの要素を切り出し，6 行目で reg の記述するパターンに一致する要素のみを抽出し，これを変数 items に格納している．この JavaScript のプログラムを書いた開発者は，6 行目で悪意のある文字列が取り除かれること（サニタイズ）を意図している．例えば，parts が [#, compare, iPhone17, <script>, Pixel11] の場合には，<script> が取り除かれ，items は [iPhone17, Pixel11] となることを期待している．

最後に，7 行目では items の各要素を文字列 ␣vs␣ で連結し，その文字列を DOM 要素 titleElem に HTML として挿入する（HTML がレンダリングされる）．例えば，items が [iPhone17, Pixel11] なら，HTML として iPhone17 vs Pixel11 が挿入され，ページ内にその文字列が表示される．

ところが，先述の通りこのプログラムにはバグがある．4 行目にて定義されている正規表現の範囲文字 [0-9a-z%-␣.] にバグがあり，ASCII コードにおける多くの記号を許容するようになっている．すなわち，%-_ となっている箇所に問題がある．開発者の意図としては，%，-，または _ のいずれか 1 文字であることを表現しているが，実際には - が文字の範囲指定として解釈され，ASCII テーブルにおける %（37 番）から _（95 番）までのいずれか 1 文字であることを表現してしまっている．そのため，攻撃に利用される文字である <（60 番）や '（39 番）などがこれらに一致してしまい，取り除かれない．結果として，この場合，URL の hash に #/compare/<img src=1 onerror=alert('hack')> と入力すると，シンクに悪意のあるコードを含む文字列である <img src=1 onerror=alert('hack')> を到達させることができる．

2.2 DOM 型 XSS 検知

既存の DOM 型 XSS 検知技術では，テイント解析技術と文字列制約解消技術を組み合わせることでこの検知を試みる．すなわち，既存技術ではまずテイント解析技術を用いて事前に定めたソースから取得した値にテイントと呼ばれる値を付与し，以降の処理ではテイントが付与された値に操作が適用されるたびにその操作をテイントに紐づけて記録する．そして，シンクにテイントの付与された値が渡されたとき，これは外部入力がシンクに到達したことを表すため，これを検知し解析を行う．解析には文字列制約解消技術を用いる．文字列制約解消技術とは文字列に関する制約が充足可能かどうか判定する技術であり [4]，ここではテイントに記録された操作を対応する文字列制約に変換し，これに攻撃用の文字列がシンクに到達するかどうかを確認する制約を追加した上で充足可能性を判定する．これが充足可能であった場合，DOM 型 XSS の脆弱性が存在するものと判断し，得られた入力の変数に対する割り当てがその脆弱性を悪用する文字列を表す．

次に，既存技術における課題を説明する．多くの既存研究ではテイント解析で取得するログは主に文字列から文字

[*2] https://developer.mozilla.org/ja/docs/Web/API/Location/hash

© 2026 Information Processing Society of Japan

列への関数（例えば，文字列変換に用いられる replace 関数や前後の空白を削除する trim 関数など）に限定される．そのため，第 2.1 章で説明したような文字列をリストに変換して加工した後に文字列に変換する処理がソースからシンクまでのパスで行われた場合，リストに関する操作はログに記録されない．現に，先行研究 [20] のツールである Foxhound[*3] を用いて第 2.1 章のプログラムを評価すると，リストに対する操作である slice 関数や filter 関数，正規表現を用いたパターンマッチングを行う test 関数，リストから要素を取得する添字による参照 x[n] はログに記録されない．[*4] そのため，解析を行う際にはこれらが正確に解析されず，保守的に近似された制約が構築され，実際に脆弱性を悪用できる攻撃用の文字列を得ることは難しい．そこで，本研究では，この課題がインジェクション検知に与える影響を明らかにすべく，テイント解析部分を改良することでこれらの操作を取得可能にし，世の中のウェブアプリケーションにそれらの操作がどれだけ利用されているのか調査を行う．

3. 手法

本章では，実世界のウェブアプリケーションで利用されている正規表現及び操作の収集方法を説明する．正規表現の収集には既存研究の手法 [20] をそのまま用いる．具体的には，ウェブブラウザに対して次のような拡張を施す．まず，文字列型の変数にテイントと呼ばれる情報を保持するように拡張する．そして，ブラウザ内で文字列型の変数に対する操作を行うたびにテイントを更新した上で操作後の文字列変数にも伝播するように拡張する．この手法を用いることでウェブアプリケーション内におけるソース及びシンク間の文字列操作を記録することができるため，それらの操作で用いられた正規表現を収集することで実世界で利用されている正規表現の実態を探ることができる．ところが，この方法では取得できる関数が事前に定められており，その多くは文字列から文字列への関数に限られるため，文字列からリストへの操作やリストから文字列への操作については取得できない場合が多い．そこで，以降ではより簡易的に様々な操作を取得する方法を説明する．

3.1 実世界で利用される操作の収集方法

本章では，ソースから入力された値がシンクに到達するまでの間に適用された操作を取得する手法を説明する．**図 1** に本手法の概要を示す．本手法は入力として検査対象のウェブアプリケーションの URL を受け取り，出力としてそのウェブアプリケーション内にある**フロー**の集合を返す．フローとは，ソースから入力された値がシンクに到達するまでの間に適用された**操作**の列を指す．ここで，操作とは関数名やそのパラメータからなる組である．例えば，2 章で示した例の場合，取得されるフローは [(split, /), (slice, 2), (filter, p), (join, vs)] のようになる．ここで，p は filter の述語である (part) => reg.test(part) を表す．

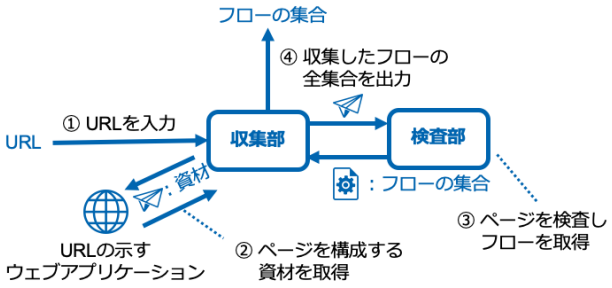


図 1 操作取得手順

まず，収集部では与えられた URL を起点にウェブアプリケーションにアクセスし，そのページを表示するために必要な資材を全て取得する．ここは通常のウェブアプリケーションへのアクセスと同様に行う．次に，取得した資材を検査部に渡す．検査部では資材を受け取ると通常の方法で資材からウェブページを構築する．構築が完了した後，インターネットへの通信を遮断した上で構築したウェブページ内に含まれる各ソースに対して入力を与え，ウェブアプリケーションを動かす．ここで，本評価では DOM 型 XSS を対象としているため，外部との通信が行えないことによる影響はない．すなわち，通信が行えないことによりシンクに到達できないようなものがあったとしてもそれらは DOM 型 XSS にはなり得ないため，それらが収集対象から除外されても DOM 型 XSS に関わるフローを見逃すことはない．シンクに到達したものがあった場合には，そこで得られたフローを記録しておき，全シンクの調査が完了した後にそれらのフローの集合を収集部に返す収集部では，検査部からフローの集合を受け取るとインターネットとの通信を再開する．そして，事前に定められた深さの範囲内でウェブアプリケーションの探索を行い，その都度上記の方法を用いて取得したウェブページの検査を行いフローの集合を取得する．その間取得したフローの集合は記録を続け，指定した深さ内での探索が完了した後に記録したフロー全体の集合を出力して停止する．

4. 評価

実装　テイント解析技術の実装に関して，テイントの伝播・付与を実現する方法としてはブラウザを改良する方法 [14], [20], [21], [23], [26] や，プロキシなどを経由して JavaScript を動的に書き換えテイント追跡機能を付与する方法 [25] がある．実世界のウェブアプリケーションで利

[*3] https://github.com/SAP/project-foxhound
[*4] split に関しては記録されている．これは，split の直後に join が続く場合は replace 関数に書き換えることができるため，部分的にサポートしていることが [20] にて言及されている．

© 2026 Information Processing Society of Japan

表 1 文字列および文字列リストに対する操作

操作の分類	メソッド
文字列 → 文字列リスト	`split`, `match`, `matchAll`
文字列リスト → 文字列リスト	`map`, `filter`, `slice`
文字列リスト → 文字列	`join`, `reduce`
文字列 → 整数値	`indexOf`, `lastIndexOf`, `search`, `charCodeAt`
文字列 → ブール値	`test`, `includes`, `startsWith`, `endsWith`

表 2 ソースおよびシンクの一覧

ソース	シンク
`location.hash`	`document.write`
`location.href`	`document.writeln`
`location.search`	`Element.innerHTML`
`location.pathname`	`Element.outerHTML`
`location.host`	`insertAdjacentHTML`
`document.referrer`	`eval`
`window.name`	`new Function(···)`
`document.cookie`	`setTimeout`
`localStorage`	`script.text`
`postMessage`	`script.src`
	`style.cssText`

表 3 各演算子の出現件数（全体：1,107,252 件）

演算子	出現件数	割合
肯定先読み (`?=`···)	14,530	1.31%
否定先読み (`?!`···)	26,086	2.36%
肯定後読み (`?<=`···)	9,522	0.86%
否定後読み (`?<!`···)	627	0.06%
後方参照 `\i`	5,537	0.5%
量指定子 ···{i, j}	36,946	3.34%
Greedy な繰り返し ···*	604,605	54.6%
Lazy な繰り返し ···*?	20,466	1.85%
Greedy なオプション ···?	69,703	6.3%
Lazy なオプション ···??	0	0%

用される正規表現の収集には前者を用い，操作の収集には後者を用いた．正規表現の収集には既存手法が実装された OSS のブラウザである Foxhound[*5]を用い，操作の収集には第 3.1 章で説明した方法を実装したツールを用いた．操作の収集に関して，ウェブアプリケーション内を探索する際の深さは 1 に設定した．また，本ツールが対象とした取得対象の操作の一覧を**表 1** に，ソース及びシンクの一覧を**表 2** に示す．

4.1 正規表現に関する調査

本章では，既存手法を用いて実世界のウェブアプリケーション内で利用されている正規表現を収集した結果を報告する．収集した結果を**表 3** に示す．正規表現の収集には世界中からよくアクセスされているサイトの一覧である Tranco リスト[*6]を用い，そこに記載があるウェブアプリケーションを対象に Foxhound を用いてフローを収集した．時間の都合から全件を対象とした評価はできなかったため，リスト内から合計 3,957 件を対象に評価を行った．その中から検出されたフローの総件数は 1,880,262 件であり，3,957 件中 3,760 件 (約 95%) のウェブアプリケーションでは少なくとも 1 件のフローが検出された．フロー全体の中に出現する正規表現は 1,168,177 件であった．

出現する正規表現に関する分析は次の通りである．まず，先読み及び後読みに関して，これらはフローの中に少なからず出現しているものの，それらの形はある程度限定されたものであることを確認した．すなわち，先読み・後読みの中に再度先読み・後読みが出現するようなネスト構造を持つものは存在しなかった．また，それらの中の表現に後方参照は出現しなかった．一方で，UTF-16 の文字コードを持つ範囲文字からなる先読み・後読みが存在した．例えば，(`?<![\uD800-\uDBFF]`) のような正規表現を確認した．このような表現を素朴にオートマトンとして記述した場合，1 つの範囲文字の遷移であってもオートマトン上では非常に多くの遷移を用いて表現することになる．そのため，これらを精緻に，かつ現実的な時間内で検証するには Symbolic Finite Automata（SFA） [13] などの拡張を施したモデルを用いる必要がある．UTF-16 の文字コードを持つ範囲文字は先読み・後読みの内部の表現に関わらず多くの箇所で出現していることを確認した．そのため，実用上は SFA のような文字の範囲を効率的に扱うことのできる拡張を施したモデルを用いることが重要な場面が少なからず存在している．また，先読みや後読みの内部の表現は比較的サイズ[*7]の小さいものが多く（例えば，(`?![0-9a-fA-F]{2}`) や (`?<=([^0-9-.]|^)`) など），この点を活かした検証方法を考える余地があることを確認した．一部には文字列が複数含まれるややサイズの大きいものも確認した（例えば，(`?!area|br|col|embed|···|link|meta|param`) など）．しかし，それらは複数の文字列を選択で繋げた形のものであり，そのような形に特化した検証方法を考える余地がある．

次に，後方参照に関する調査結果を述べる．一般に，後方

[*5] `https://github.com/SAP/project-foxhound`

[*6] `https://tranco-list.eu/`

[*7] ここで，正規表現のサイズとは正規表現に対応する抽象構文木の頂点数を指す．

© 2026 Information Processing Society of Japan

参照を含む正規表現を対象とした検証問題（例えば，type checking）は容易に決定不能になる [11]．しかし，本調査で確認した後方参照の使い方はいずれも制限されたものばかりであり，この制限を活かすことで検証問題を決定可能にする余地は残されている．具体的には，後方参照に対応するキャプチャグループは Kleene star などの繰り返しの演算子の中には出現しなかった．例えば，本調査では後方参照を含む正規表現 `^([^;]*)(;\1)+` が利用されていることを確認した．ここで，後方参照 `\1` に対応するキャプチャグループ `([^;]*)` は内部には Kleene star が含まれるものの，それ自体は Kleene star に囲まれていない．また，後方参照に対応するキャプチャグループの内部の表現はいずれもサイズが小さいものばかりであった．例えば，上記の正規表現の他には次の正規表現 `(\w+)\1*` が利用されていることを確認した．この正規表現も，先ほどの例と同様に後方参照に対応するキャプチャグループの内部の表現は範囲文字に対する Kleene star からなる表現であり，そのサイズは小さい．

最後に，量指定子に関する調査結果を述べる．量指定子は下限や上限に大きな定数が指定された場合に，それを素朴に展開してオートマトンとすると状態数が膨大になる．そのため，オートマトンに工夫を施すことで状態数が急増することを避け，実用上効率的にマッチングを行う方法が検討されている [27]．同様の問題が検証においても発生しうる．本調査では，量指定子の下限の最大値は 32 であった．具体的には，次の正規表現 `^[␣\t]*([0-9a-f]{32})?-?([0-9a-f]{16})?-?([01])?[␣\t]*$` が下限の最大値を持つものであった．このような量指定子を素朴に展開するとオートマトンの状態数が非常に大きくなりうるため，状態数の爆発を抑えつつ実用上効率的に検証を行う方法を検討する必要があることを確認した．本調査の範囲では，量指定子の対象となる表現のサイズは比較的小さいものばかりであったため，この点を活かし状態数の爆発を抑える方法が考えられるかもしれない．

4.2 操作に関する調査結果

本章では，第 3.1 章で紹介した手法を用いて実世界のウェブアプリケーションで利用される操作を収集した結果を報告する．本実験では，Tranco リストからランダムに選択した 10 件のウェブアプリケーションを対象にフローの収集を行った．収集したフローに含まれた操作（既存技術がサポートしている文字列から文字列への操作は除く）の一覧を**表 4** に示す．この表に示される操作は後述の通り一部の場合を除いていずれも既存手法では扱われていない．

既存手法 [20] では `String.prototype.split` の直後に `Array.prototype.join` が続くものに関してはその操作を `replace` 関数とみなすことができるため扱えるものとしているが，本評価においては `String.prototype.split` の直後に `Array.prototype.join` が続くことは多くなく，既存手法ではそれらを扱うことができないことを確認した．この状況への改善策として，脆弱性の有無を検証する際に既存手法では独自に実装した文字列関数による正規言語の逆像計算を用いているが，ここを既存の文字列制約解消ソルバーに置き換えるという方法が考えられる．最先端の文字列制約解消ソルバー [19] であれば，文字列からリストへの操作やリストから文字列への操作を一部サポートしているため，それらに関しては扱うことができる．ただし，そのソルバーを用いた場合であっても表 4 に掲載されている全ての操作を扱える訳ではない．例えば，`Array.prototype.map` や `Array.prototype.reduce` はサポートされておらず，`Array.prototype.filter` に関してはサポートされているものの，そのパラメタには正規表現による所属判定を行う述語しか記述できない．そのため，これらの差を完全に埋めるためには新たな検証手法が必要となる．

表 4 各操作の出現回数（全体:101,178 件）

操作	出現回数
`RegExp.prototype.test`	10,701 (10.58%)
`String.prototype.match`	10,274 (10.15%)
`String.prototype.includes`	8,726 (8.62%)
`String.prototype.indexOf`	5,387 (5.32%)
`String.prototype.split`	3,827 (3.78%)
`String.prototype.lastIndexOf`	2,728 (2.7%)
`Array.prototype.join`	1,374 (1.34%)
`Array.prototype.slice`	588 (0.58%)
`String.prototype.startsWith`	505 (0.5%)
`String.prototype.charCodeAt`	278 (0.27%)
`String.prototype.endsWith`	133 (0.13%)
`Array.prototype.filter`	12 (0.01%)
`String.prototype.search`	12 (0.01%)
`Array.prototype.reduce`	8 (0.01%)
`Array.prototype.map`	2 (0.001%)

以上をまとめると，実世界では外部入力に対して文字列からリストへの操作やリストから文字列への操作など，既存手法では扱っていない操作を少なからず含むことを確認した．そのため，既存研究によるフローの取得ではこれらが見落とされ，正確にインジェクション脆弱性を検知できない場面が存在する．

5. 関連研究

DOM 型 XSS に関する実態調査の研究は広く行われている [20], [21], [23], [26]．これらの研究では，ウェブブラウザに対してテイントを付与・追跡する機能を追加し，ソースから入力された文字列がシンクに到達するかどうか，到達した場合，そのフローの脆弱性の悪用を可能とするものかどうか検証する．しかし，これらの方法では本調査で確認した拡張機能を含む正規表現や文字列から文字列への関

© 2026 Information Processing Society of Japan

数以外の操作については扱うことができていない．

インジェクション脆弱性を検知するため，文字列制約ソルバーに関する研究が盛んに行われている [1], [2], [3], [5], [6], [7], [8], [9], [12], [16], [17], [18], [22], [28]．これらの研究では，文字列から文字列への関数は広くサポートしており，最先端のものでは文字列から文字列への関数に限らず，実世界の文字列操作の中で利用されるその他の型の関数についても扱えるように改良が重ねられている．例えば，Ostrich^{Seq} [19] では文字列リストに関する制約もいくつかサポートしている．他にも，Z3-Noodler [10] では文字列操作に伴う文字列上の位置に関する制約も扱えるように拡張が研究されている．しかし，第 4 章でも言及した通り，これらの研究でもまだ文字列リスト等を伴う文字列操作に関しての対応は十分ではない．例えば，既存研究では基本的には拡張機能を含む正規表現を伴う操作を扱うことができない．先読みと後読みを含む正規表現を制約として扱うことができる文字列制約ソルバーとして Black Ostrich [15] が存在するものの，本アイデアを実装したツールである Ostrich では正規言語の所属判定に関する制約の中でのみ先読み及び後読みの利用を許しており，文字列操作関数を表す制約の中で先読み及び後読みを扱うことはできていない．

6. 倫理的配慮

本章では，第 3.1 章で紹介したウェブサイトから操作を収集する手法に関する倫理的配慮について議論する．この手法では，ウェブアプリケーションに対して文字列を入力して実行するなどの操作を行っている．入力する文字列には攻撃に用いられるようなものは用いていないものの，その文字列に適用される操作の結果として最終的に意図せず攻撃ともとれる文字列に書き変わる可能性がある．そのような場合であってもウェブアプリケーションに対して影響を与えないための倫理的配慮を施している．具体的には，本評価ではクライアント側での操作のみが興味の対象である点を活かし，ウェブサイトの読み込みが完了した後に通信を遮断し，その状態で操作を行なっている．そのため，万が一攻撃とも取れる形の文字列がプログラムとして解釈され実行された場合であってもサーバ側には影響を与えないと考えている．また，ウェブアプリケーション内を探索する際には robots.txt を確認し，その指示に従っている．

7. おわりに

本論文では実世界に潜むインジェクション脆弱性検知技術の確立に向けて，ウェブアプリケーション内のソース・シンク間で利用される操作に関する実態調査を行った．具体的には，クライアント側で調査を完結することができる DOM 型 XSS を対象に，ソースからシンク間で利用される操作を収集し，そこに含まれる正規表現や操作に関する分析を行った．結果として，実世界の操作には既存手法ではサポートされていない拡張機能を含む正規表現がしばしば利用されていることを確認し，それらの拡張機能がある種の限定された形で利用されていることが多いことも確認した．加えて，外部入力には必ずしも文字列から文字列への関数だけが適用される訳ではなく，文字列からリストへの操作やリストに対する操作，リストから文字列に戻す操作など既存手法では考慮されていない様々な種類の操作が適用されることを確認した．そのため，実世界のインジェクション脆弱性を正確に検知するにはそれらの操作も正確に捉える必要がある．

今後は本調査結果を踏まえ，実世界のインジェクション脆弱性を検知する上で不足している技術的課題の解決を試みることを検討している．また，本研究では主に技術的な観点での不足を調査したが，実世界からインジェクション脆弱性を根絶するには確立した技術を普及させることも重要である．それには技術としての利用のしやすさ・導入のしやすさも重要となるため，この点についても今後検討を進める予定である．

参考文献

[1] Abdulla, P. A., Atig, M. F., Chen, Y.-F., Holík, L., Rezine, A., Rümmer, P. and Stenman, J.: String Constraints for Verification, *Computer Aided Verification* (Biere, A. and Bloem, R., eds.), Cham, Springer International Publishing, pp. 150–166 (2014).

[2] Abdulla, P. A., Atig, M. F., Chen, Y.-F., Holík, L., Rezine, A., Rümmer, P. and Stenman, J.: Norn: An SMT Solver for String Constraints, *Computer Aided Verification* (Kroening, D. and Păsăreanu, C. S., eds.), Cham, Springer International Publishing, pp. 462–469 (2015).

[3] Abdulla, P. A., Faouzi Atig, M., Chen, Y.-F., Diep, B. P., Holík, L., Rezine, A. and Rümmer, P.: Trau: SMT solver for string constraints, *2018 Formal Methods in Computer Aided Design (FMCAD)*, pp. 1–5 (online), DOI: 10.23919/FMCAD.2018.8602997 (2018).

[4] Amadini, R.: A Survey on String Constraint Solving, *ACM Comput. Surv.*, Vol. 55, No. 1 (online), DOI: 10.1145/3484198 (2021).

[5] Blahoudek, F., Chen, Y.-F., Chocholatý, D., Havlena, V., Holík, L., Lengál, O. and Síč, J.: Word equations in synergy with regular constraints (extended version), *Constraints*, Vol. 30, No. 1–4, p. 1–34 (online), DOI: 10.1007/s10601-025-09379-w (2025).

[6] Chen, T., Flores-Lamas, A., Hague, M., Han, Z., Hu, D., Kan, S., Lin, A. W., Rümmer, P. and Wu, Z.: Solving string constraints with Regex-dependent functions through transducers with priorities and variables, *Proc. ACM Program. Lang.*, Vol. 6, No. POPL (online), DOI: 10.1145/3498707 (2022).

[7] Chen, T., Hague, M., He, J., Hu, D., Lin, A. W., Rümmer, P. and Wu, Z.: A Decision Procedure for Path Feasibility of String Manipulating Programs with Integer Data Type, *Automated Technology for Verification and Analysis: 18th International Symposium, ATVA*

© 2026 Information Processing Society of Japan

2020, Hanoi, Vietnam, October 19–23, 2020, Proceedings, Berlin, Heidelberg, Springer-Verlag, p. 325–342 (online), DOI: 10.1007/978-3-030-59152-6_18 (2020).

[8] Chen, T., Hague, M., Lin, A. W., Rümmer, P. and Wu, Z.: Decision procedures for path feasibility of string-manipulating programs with complex operations, *Proc. ACM Program. Lang.*, Vol. 3, No. POPL (online), DOI: 10.1145/3290362 (2019).

[9] Chen, Y.-F., Chocholatý, D., Havlena, V., Holík, L., Lengál, O. and Síč, J.: Z3-Noodler: An Automata-based String Solver, *Tools and Algorithms for the Construction and Analysis of Systems: 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6–11, 2024, Proceedings, Part I*, Berlin, Heidelberg, Springer-Verlag, pp. 24–33 (online), DOI: 10.1007/978-3-031-57246-3_2 (2024).

[10] Chen, Y.-F., Havlena, V., Hečko, M., Holík, L. and Lengál, O.: A Uniform Framework for Handling Position Constraints in String Solving, *Proc. ACM Program. Lang.*, Vol. 9, No. PLDI (online), DOI: 10.1145/3729273 (2025).

[11] Chida, N. and Terauchi, T.: On Lookaheads in Regular Expressions with Backreferences, *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022)* (Felty, A. P., ed.), Leibniz International Proceedings in Informatics (LIPIcs), Vol. 228, Dagstuhl, Germany, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, pp. 15:1–15:18 (online), DOI: 10.4230/LIPIcs.FSCD.2022.15 (2022).

[12] Chocholatý, D., Havlena, V., Holík, L., Síč, J. and Šedý, M.: String Solving with Stabilization and Transducers, *Computer Aided Verification: 38th International Conference, CAV 2026, Lisbon, Portugal, July 26–29, 2026, Proceedings, Part II*, Berlin, Heidelberg, Springer-Verlag, p. 50–74 (online), DOI: 10.1007/978-3-032-32526-6_3 (2026).

[13] D'Antoni, L. and Veanes, M.: The Power of Symbolic Automata and Transducers, *Computer Aided Verification* (Majumdar, R. and Kunčak, V., eds.), Cham, Springer International Publishing, pp. 47–67 (2017).

[14] Di, P.: DominatorPro: Securing Next Generation of Web Applications, URL `https://dominator.mindedsecurity.com/` (2012).

[15] Eriksson, B., Stjerna, A., De Masellis, R., Rüemmer, P. and Sabelfeld, A.: Black Ostrich: Web Application Scanning with String Solvers, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, New York, NY, USA, Association for Computing Machinery, p. 549–563 (online), DOI: 10.1145/3576915.3616582 (2023).

[16] Felmetsger, V., Cavedon, L., Kruegel, C. and Vigna, G.: Toward automated detection of logic vulnerabilities in web applications, *Proceedings of the 19th USENIX Conference on Security*, USENIX Security'10, USA, USENIX Association, p. 10 (2010).

[17] Havlena, V., Hečko, M., Holík, L. and Lengál, O.: Negated String Containment Is Decidable, *50th International Symposium on Mathematical Foundations of Computer Science (MFCS 2025)* (Gawrychowski, P., Mazowiecki, F. and Skrzypczak, M., eds.), Leibniz International Proceedings in Informatics (LIPIcs), Vol. 345, Dagstuhl, Germany, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, pp. 56:1–56:20 (online), DOI: 10.4230/LIPIcs.MFCS.2025.56 (2025).

[18] Holík, L., Janků, P., Lin, A. W., Rümmer, P. and Vojnar, T.: String constraints with concatenation and transducers solved efficiently, *Proc. ACM Program. Lang.*, Vol. 2, No. POPL (online), DOI: 10.1145/3158092 (2017).

[19] Hu, D., Chen, T., Rümmer, P., Song, F. and Wu, Z.: Decision Procedure for a Theory of String Sequences, *Programming Languages and Systems* (Potanin, A., ed.), Singapore, Springer Nature Singapore, pp. 217–238 (2026).

[20] Klein, D., Barber, T., Bensalim, S., Stock, B. and Johns, M.: Hand Sanitizers in the Wild: A Large-scale Study of Custom JavaScript Sanitizer Functions, *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*, pp. 236–250 (online), DOI: 10.1109/EuroSP53844.2022.00023 (2022).

[21] Lekies, S., Stock, B. and Johns, M.: 25 million flows later: large-scale detection of DOM-based XSS, *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, CCS '13, New York, NY, USA, Association for Computing Machinery, pp. 1193—1204 (online), DOI: 10.1145/2508859.2516703 (2013).

[22] Li, G. and Ghosh, I.: PASS: String Solving with Parameterized Array and Interval Automaton, *Hardware and Software: Verification and Testing* (Bertacco, V. and Legay, A., eds.), Cham, Springer International Publishing, pp. 15–31 (2013).

[23] Melicher, W., Das, A., Sharif, M., Bauer, L. and Jia, L.: Riding out DOMsday: Towards Detecting and Preventing DOM Cross-Site Scripting, *Network and Distributed System Security Symposium*, (online), available from ⟨https://api.semanticscholar.org/CorpusID:3389782⟩ (2018).

[24] National Institute of Standards and Technology (NIST): NVD Vulnerability Statistics, `https://nvd.nist.gov/vuln/search/statistics` (2026).

[25] Parameshwaran, I., Budianto, E., Shinde, S., Dang, H., Sadhu, A. and Saxena, P.: DexterJS: robust testing platform for DOM-based XSS vulnerabilities, *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, New York, NY, USA, Association for Computing Machinery, p. 946–949 (online), DOI: 10.1145/2786805.2803191 (2015).

[26] Sabino, N., Cassel, D., Abreu, R., Adão, P., Bauer, L. and Jia, L.: DOM-XSS Detection via Webpage Interaction Fuzzing and URL Component Synthesis, *Proceedings of the 33rd Network and Distributed System Security Symposium*, Internet Society, (online), DOI: 10.14722/ndss.2026.231467 (2026).

[27] Turoňová, L., Holík, L., Lengál, O., Saarikivi, O., Veanes, M. and Vojnar, T.: Regex matching with counting-set automata, *Proc. ACM Program. Lang.*, Vol. 4, No. OOPSLA (online), DOI: 10.1145/3428286 (2020).

[28] Zheng, Y., Ganesh, V., Subramanian, S., Tripp, O., Berzish, M., Dolby, J. and Zhang, X.: Z3str2: an efficient solver for strings, regular expressions, and length constraints, *Form. Methods Syst. Des.*, Vol. 50, No. 2–3, p. 249–288 (online), DOI: 10.1007/s10703-016-0263-6 (2017).

© 2026 Information Processing Society of Japan